CODSOFT Artificial Intelligence Internship

SMART TIC TAC TOE CHALLENGE (Human vs Computer)

> **SUBMITTED BY: -** Khushi Maina

> **INTERNSHIP DOMAIN: -** Artificial Intelligence

> **ORGANIZATION: -** CODSOFT

Introduction

Tic Tac Toe is a popular two-player strategy game played on a 3×3 grid. Players take turns marking spaces in the grid with symbols. One player uses "X" and the other uses "O". The winner is the player who succeeds in placing three of their marks in a horizontal, vertical, or diagonal row.

This project allows a human player to play against a computer. The computer automatically selects available positions on the board and competes with the player until a winner is found or the match ends in a draw.

Objective

The objective of this project is to develop a console-based Tic Tac Toe game in Python where a human player competes against a computer-controlled opponent. The project demonstrates the use of Python programming concepts such as functions, loops, conditional statements, lists, exception handling, and random number generation

Software Requirements

- Python 3.x
- Any Python IDE (IDLE, VS Code, PyCharm, etc.)
- Windows/Linux/Mac Operating System

Functional Requirements

1. Display the Tic Tac Toe board.
2. Allow the user to enter a position from 1 to 9.
3. Validate user input.

4. Prevent overwriting occupied positions.
5. Generate computer moves automatically.
6. Detect winning conditions.
7. Detect draw situations.
8. Allow replaying the game.

System Design

Main Components

1. Board Initialization
2. Display Functions
3. Player Move Function
4. AI Move Function
5. Winner Checking Function
6. Draw Checking Function
7. Game Loop

Algorithm

Step 1

Initialize a 3×3 game board using a list.

Step 2

Display board positions for user reference.

Step 3

Accept user input and validate it.

Step 4

Place "X" on the selected position.

Step 5

Check if the player has won.

Step 6

If not won, let the AI choose a random empty position.

Step 7

Place "O" on the selected position.

Step 8

Check if the AI has won.

Step 9

Check whether the board is full.

Step 10

Declare Draw if no empty positions remain.

Step 11

Ask the user whether to play again.

Modules Used

Random Module

The random module is used to generate a random move for the AI player.

Example:

```
import random

move = random.choice(empty_positions)
```

Functions Description

print_positions()

Displays the position numbers from 1 to 9.

print_board()

Displays the current board state.

check_winner(player)

Checks whether a player has achieved a winning combination.

is_full()

Checks whether the board is completely filled.

player_move()

Accepts and validates the player's move.

ai_move()

Generates a random move for the AI.

play_game()

Controls the complete game flow.

Source Code

```
print("=" * 40)
print("SMART TIC TAC TOE CHALLENGE")
print(" Human vs Computer")
print("=" * 40)
import random


board = [" " for _ in range(9)]


def print_positions():
    print("\nPositions:")
    print("1 | 2 | 3")
    print("--+---+--")
    print("4 | 5 | 6")
    print("--+---+--")
    print("7 | 8 | 9\n")


def print_board():
    print()
```

```
    for i in range(3):
        print(board[i*3] + " | " + board[i*3+1] + " | " + board[i*3+2])
        if i < 2:
            print("--+---+--")
    print()

def check_winner(player):
    win_combinations = [
        (0,1,2), (3,4,5), (6,7,8),
        (0,3,6), (1,4,7), (2,5,8),
        (0,4,8), (2,4,6)
    ]

    for combo in win_combinations:
        if board[combo[0]] == board[combo[1]] == board[combo[2]] == player:
            return True

    return False

def is_full():
    return " " not in board

def player_move():
    while True:
        try:
            move = int(input("Enter your position (1-9): ")) - 1
```

```
            if move < 0 or move > 8:
                print("Choose a number between 1 and 9.")
                continue

            if board[move] != " ":
                print("Position already occupied.")
                continue

            board[move] = "X"
            break

        except ValueError:
            print("Please enter a valid number.")

def ai_move():
    empty_positions = []

    for i in range(9):
        if board[i] == " ":
            empty_positions.append(i)

    move = random.choice(empty_positions)

    board[move] = "O"
```

```
    print("AI chose position", move + 1)

def play_game():

    print("=" * 35)
    print("TIC TAC TOE AI")
    print("=" * 35)

    print_positions()

    while True:

        print_board()

        player_move()

        if check_winner("X"):
            print_board()
            print("Excellent Move! You Defeated The Computer.!")
            break

        if is_full():
            print_board()
            print("Match Draw!")
            break
```

```
        ai_move()

        if check_winner("O"):
            print_board()
            print("AI Wins!")
            break

        if is_full():
            print_board()
            print("Match Draw!")
            break

while True:

    board = [" " for _ in range(9)]

    play_game()

    again = input("\nPlay Again? (y/n): ")

    if again.lower() != "y":
        print("Thanks for Playing!")
        break
```

Output

Figure :- 1 Menu Module

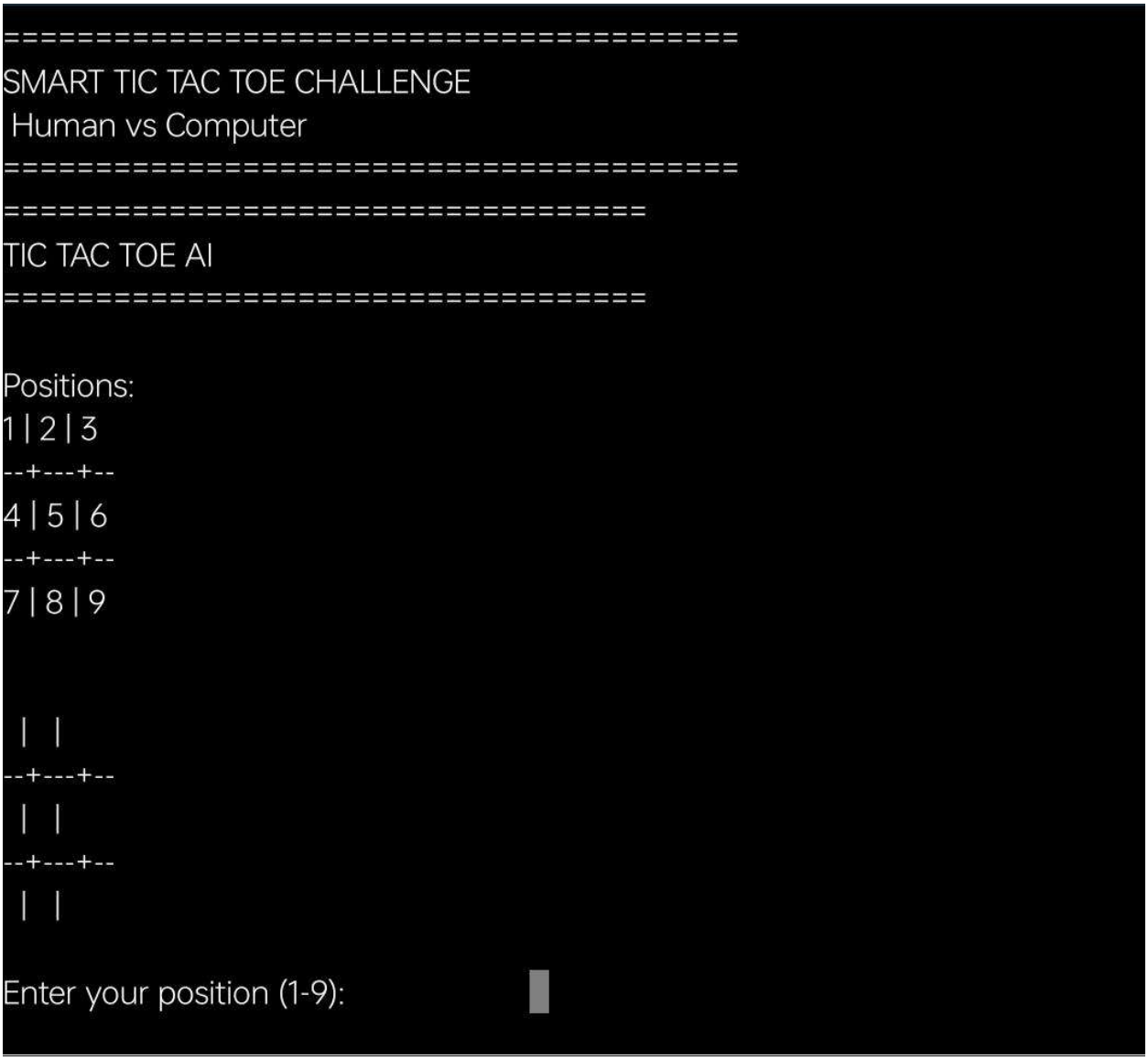
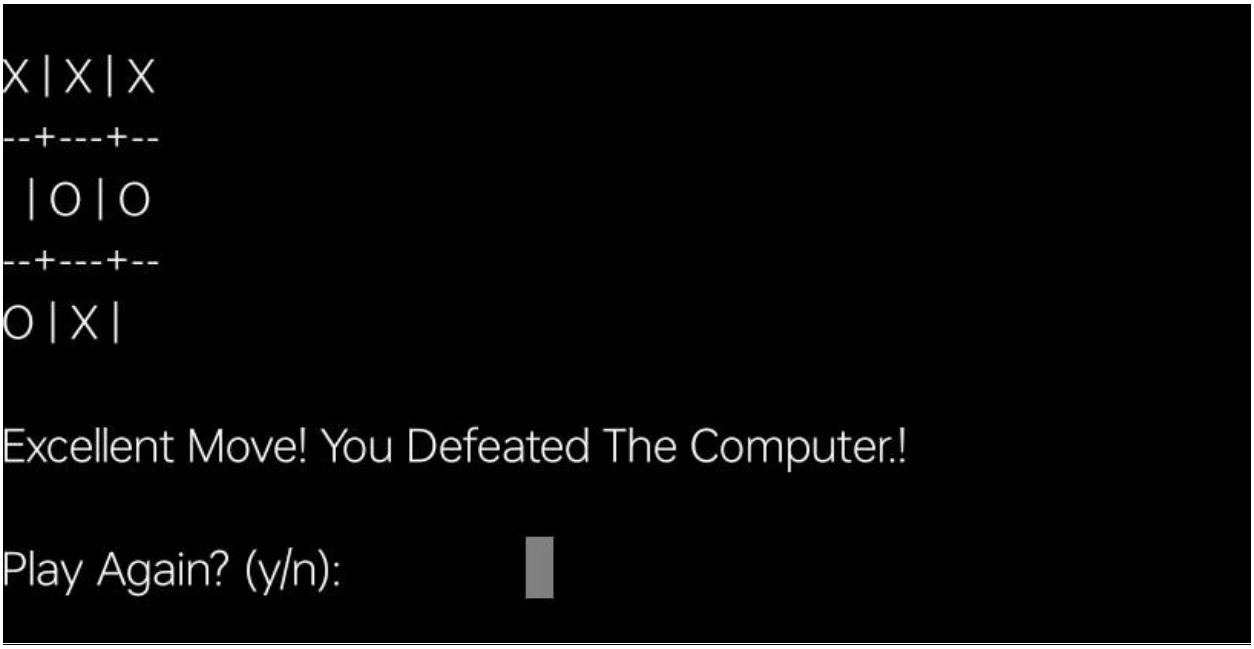


Figure:- 2 Sample



Advantages

- Easy to understand.
- Beginner-friendly project.
- Demonstrates Python fundamentals.
- Includes input validation.
- Interactive gameplay.

Limitations

- AI uses random moves only.
- AI is not fully intelligent.
- No graphical user interface (GUI).
- No score tracking system.

Future Enhancements

- Implement Minimax Algorithm for unbeatable AI.
- Add difficulty levels.
- Create a GUI using Tkinter.

- Add scoreboards.
- Add multiplayer mode.

Conclusion

The Smart Tic Tac Toe Challenge successfully implements a Human vs Computer game using Python. The project demonstrates important programming concepts including lists, loops, functions, condition checking, exception handling, and random number generation. It serves as an excellent beginner-level Python project and provides a foundation for implementing advanced AI techniques in the future.